```
import numpy as np
import matplotlib.pyplot as plt

# Define time axis
t = np.arange(-10, 11, 1)   # From -10 to 10 with step size 1
```

```
# (a) Unit Step Function
u = np.where(t >= 0, 1, 0)
```

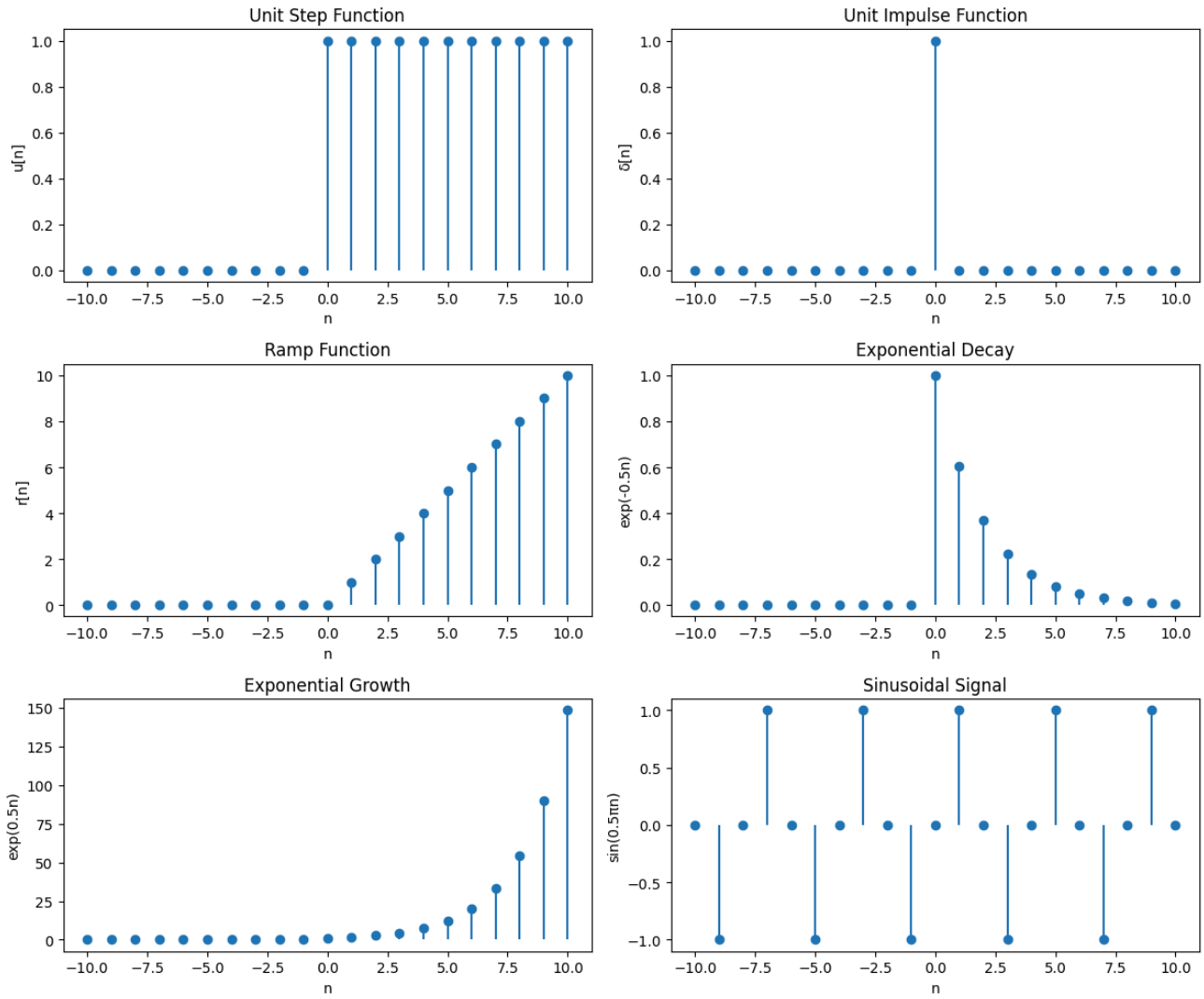
```
# (b) Unit Impulse Function (δ[n])
delta = np.where(t == 0, 1, 0)
```

```
# (c) Ramp Function
r = np.where(t >= 0, t, 0)
```

```
# (d) Exponential Signals
exp_decay = np.exp(-0.5 * t) * (t >= 0)  # Decaying exponential for t>=0
exp_grow = np.exp(0.5 * t) * (t >= 0)    # Growing exponential for t>=0
```

```
# (e) Sinusoidal Signal
sinusoidal = np.sin(0.5 * np.pi * t)
```

```
# Plotting
plt.figure(figsize=(12, 10))
# (a) Unit Step
plt.subplot(3, 2, 1)
plt.stem(t, u, basefmt=" ")
plt.title("Unit Step Function")
plt.xlabel("n")
plt.ylabel("u[n]")

# (b) Unit Impulse
plt.subplot(3, 2, 2)
plt.stem(t, delta, basefmt=" ")
plt.title("Unit Impulse Function")
plt.xlabel("n")
plt.ylabel("δ[n]")

# (c) Ramp Function
plt.subplot(3, 2, 3)
plt.stem(t, r, basefmt=" ")
plt.title("Ramp Function")
plt.xlabel("n")
plt.ylabel("r[n]")

# (d1) Exponential Decay
plt.subplot(3, 2, 4)
plt.stem(t, exp_decay, basefmt=" ")
plt.title("Exponential Decay")
plt.xlabel("n")
plt.ylabel("exp(-0.5n)")

# (d2) Exponential Growth
plt.subplot(3, 2, 5)
plt.stem(t, exp_grow, basefmt=" ")
plt.title("Exponential Growth")
plt.xlabel("n")
plt.ylabel("exp(0.5n)")

# (e) Sinusoidal
plt.subplot(3, 2, 6)
plt.stem(t, sinusoidal, basefmt=" ")
plt.title("Sinusoidal Signal")
plt.xlabel("n")
plt.ylabel("sin(0.5πn)")
plt.tight_layout()
plt.show()
```




```
# 2. (a) Generate a continuous sinusoidal signal
f = 5   # signal frequency (Hz)
t_cont = np.linspace(0, 1, 1000)  # fine time axis (continuous signal)
x_cont = np.sin(2 * np.pi * f * t_cont)
```

```
# Nyquist rate (fs >= 2f)
fs_nyquist = 2 * f
fs_above = 4 * f
fs_below = f  # below Nyquist -> aliasing
```

```
# (b) Sample at different rates
def sample_signal(fs):
    t_s = np.arange(0, 1, 1/fs)
    x_s = np.sin(2 * np.pi * f * t_s)
    return t_s, x_s
```

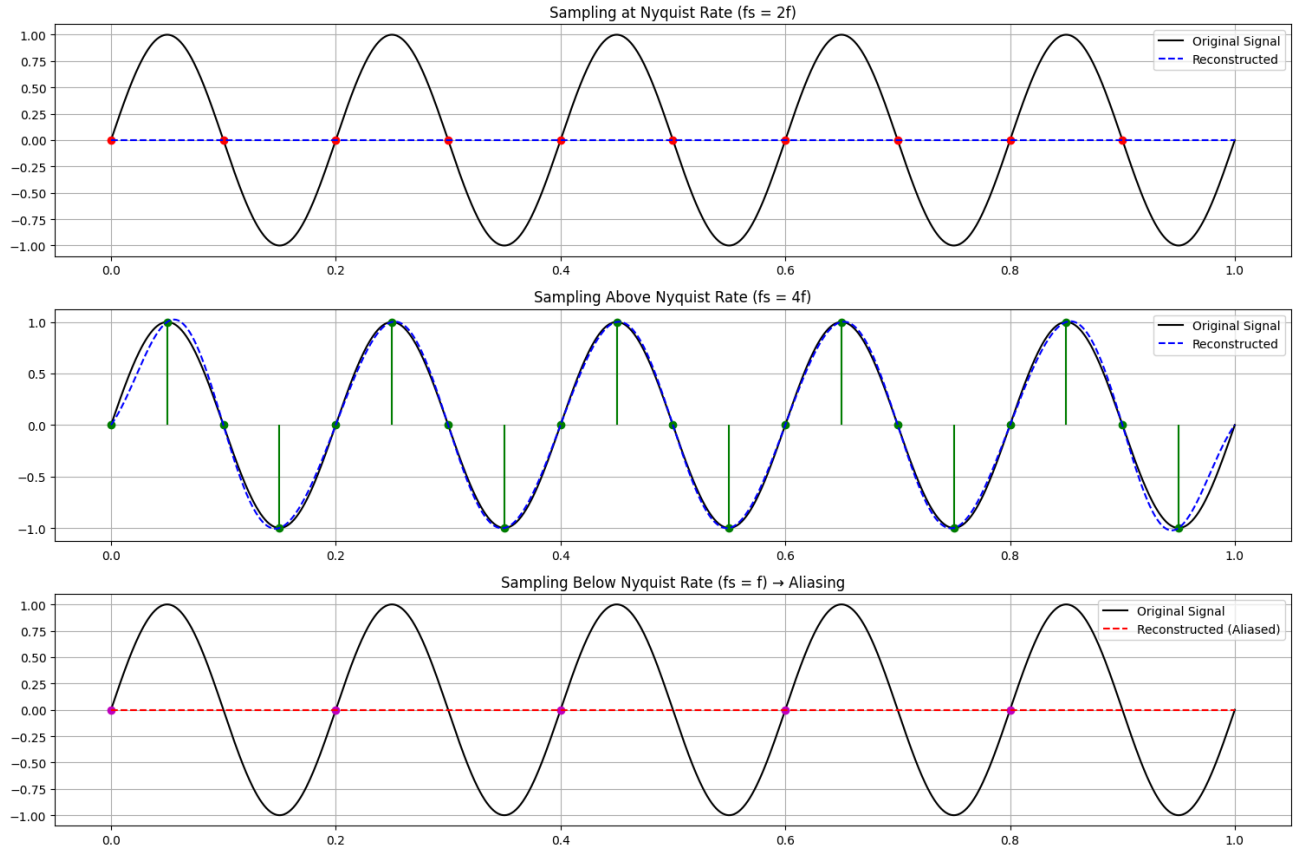
```
t_nyq, x_nyq = sample_signal(fs_nyquist)
t_above, x_above = sample_signal(fs_above)
t_below, x_below = sample_signal(fs_below)
```

```
# (c) Reconstruct the signal (using sinc interpolation for visualization)
def reconstruct(t_samples, x_samples, t):
    T = t_samples[1] - t_samples[0]
    # sinc interpolation
    x_recon = np.zeros_like(t)
    for n in range(len(x_samples)):
        x_recon += x_samples[n] * np.sinc((t - t_samples[n]) / T)
    return x_recon
```

```
x_recon_nyq = reconstruct(t_nyq, x_nyq, t_cont)
x_recon_above = reconstruct(t_above, x_above, t_cont)
```

```
x_recon_below = reconstruct(t_below, x_below, t_cont)
```

```
# (d) Plot results
plt.figure(figsize=(15, 10))

# Continuous signal + Nyquist sampling
plt.subplot(3, 1, 1)
plt.plot(t_cont, x_cont, 'k', label='Original Signal')
plt.stem(t_nyq, x_nyq, linefmt='r', markerfmt='ro', basefmt=" ")
plt.plot(t_cont, x_recon_nyq, 'b--', label='Reconstructed')
plt.title("Sampling at Nyquist Rate (fs = 2f)")
plt.legend()
plt.grid(True)

# Above Nyquist
plt.subplot(3, 1, 2)
plt.plot(t_cont, x_cont, 'k', label='Original Signal')
plt.stem(t_above, x_above, linefmt='g', markerfmt='go', basefmt=" ")
plt.plot(t_cont, x_recon_above, 'b--', label='Reconstructed')
plt.title("Sampling Above Nyquist Rate (fs = 4f)")
plt.legend()
plt.grid(True)

# Below Nyquist (Aliasing)
plt.subplot(3, 1, 3)
plt.plot(t_cont, x_cont, 'k', label='Original Signal')
plt.stem(t_below, x_below, linefmt='m', markerfmt='mo', basefmt=" ")
plt.plot(t_cont, x_recon_below, 'r--', label='Reconstructed (Aliased)')
plt.title("Sampling Below Nyquist Rate (fs = f) → Aliasing")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

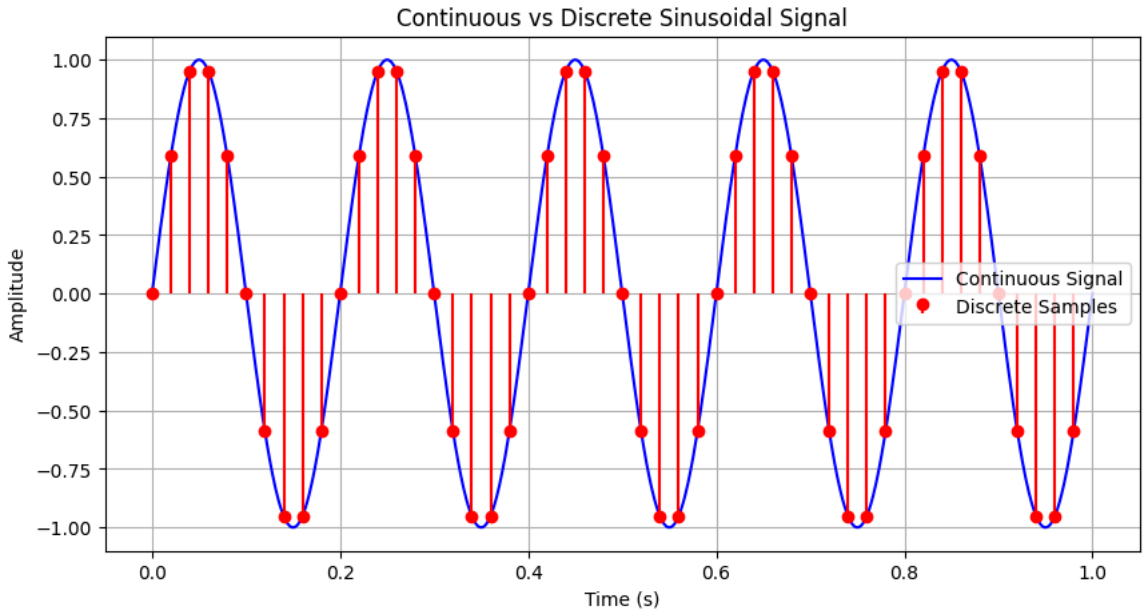



```
# Parameters
A = 1          # amplitude
f = 5          # frequency (Hz)
duration = 1   # seconds
fs = 50        # sampling frequency for discrete version (50 Hz)
```

```
# Continuous signal (high resolution time axis)
t_cont = np.linspace(0, duration, 1000)  # fine time steps
x_cont = A * np.sin(2 * np.pi * f * t_cont)
```

```
# Discrete signal (sampled version)
t_disc = np.arange(0, duration, 1/fs)
x_disc = A * np.sin(2 * np.pi * f * t_disc)
```

```
# Plotting
plt.figure(figsize=(10,5))

# Continuous signal
plt.plot(t_cont, x_cont, 'b', label="Continuous Signal")

# Discrete signal
plt.stem(t_disc, x_disc, linefmt='r', markerfmt='ro', basefmt=" ", label="Discrete Samples")

plt.title("Continuous vs Discrete Sinusoidal Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()
```




```
# Time axis
t = np.arange(-10, 11, 1)
```

```
# (a) Unit step function u(t)
u = np.where(t >= 0, 1, 0)
```

```
# (b) Time Shifting
# Delay: u(t - 3)
u_delay = np.where(t >= 3, 1, 0)
```

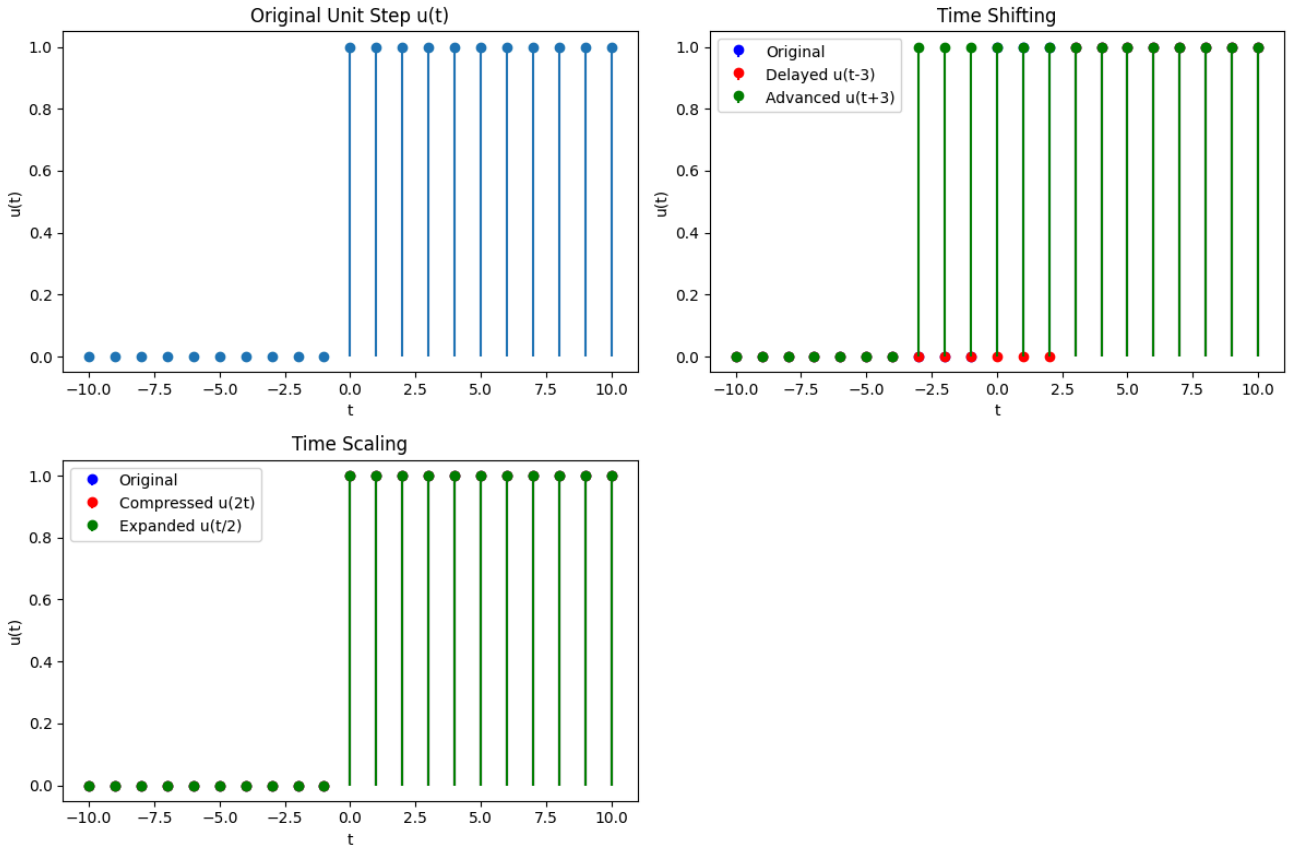
```
# Advance: u(t + 3)
u_advance = np.where(t >= -3, 1, 0)
```

```
# (c) Time Scaling
# Compression: u(2t) → step starts earlier
u_compress = np.where(2*t >= 0, 1, 0)
```

```
# Expansion: u(t/2) → step starts later
u_expand = np.where((t/2) >= 0, 1, 0)
```

```
# (d) Plotting
plt.figure(figsize=(12,8))

# Original
plt.subplot(2,2,1)
plt.stem(t, u, basefmt=" ")
plt.title("Original Unit Step u(t)")
plt.xlabel("t")
plt.ylabel("u(t)")

# Time Shifting
plt.subplot(2,2,2)
plt.stem(t, u, basefmt=" ", linefmt='b-', markerfmt='bo', label="Original")
plt.stem(t, u_delay, basefmt=" ", linefmt='r-', markerfmt='ro', label="Delayed u(t-3)")
plt.stem(t, u_advance, basefmt=" ", linefmt='g-', markerfmt='go', label="Advanced u(t+3)")
plt.title("Time Shifting")
plt.xlabel("t")
plt.legend()
plt.ylabel("u(t)")

# Time Scaling
plt.subplot(2,2,3)
plt.stem(t, u, basefmt=" ", linefmt='b-', markerfmt='bo', label="Original")
plt.stem(t, u_compress, basefmt=" ", linefmt='r-', markerfmt='ro', label="Compressed u(2t)")
plt.stem(t, u_expand, basefmt=" ", linefmt='g-', markerfmt='go', label="Expanded u(t/2)")
plt.title("Time Scaling")
plt.xlabel("t")
plt.legend()
plt.ylabel("u(t)")
```

```
plt.tight_layout()
plt.show()
```




```
# (a) Generate two sinusoidal signals
fs = 500  # Sampling frequency (for smooth plotting)
t = np.linspace(0, 1, fs)  # 1 second duration
```

```
A1, f1 = 1, 5   # Amplitude=1, Frequency=5 Hz
A2, f2 = 2, 15  # Amplitude=2, Frequency=15 Hz
```

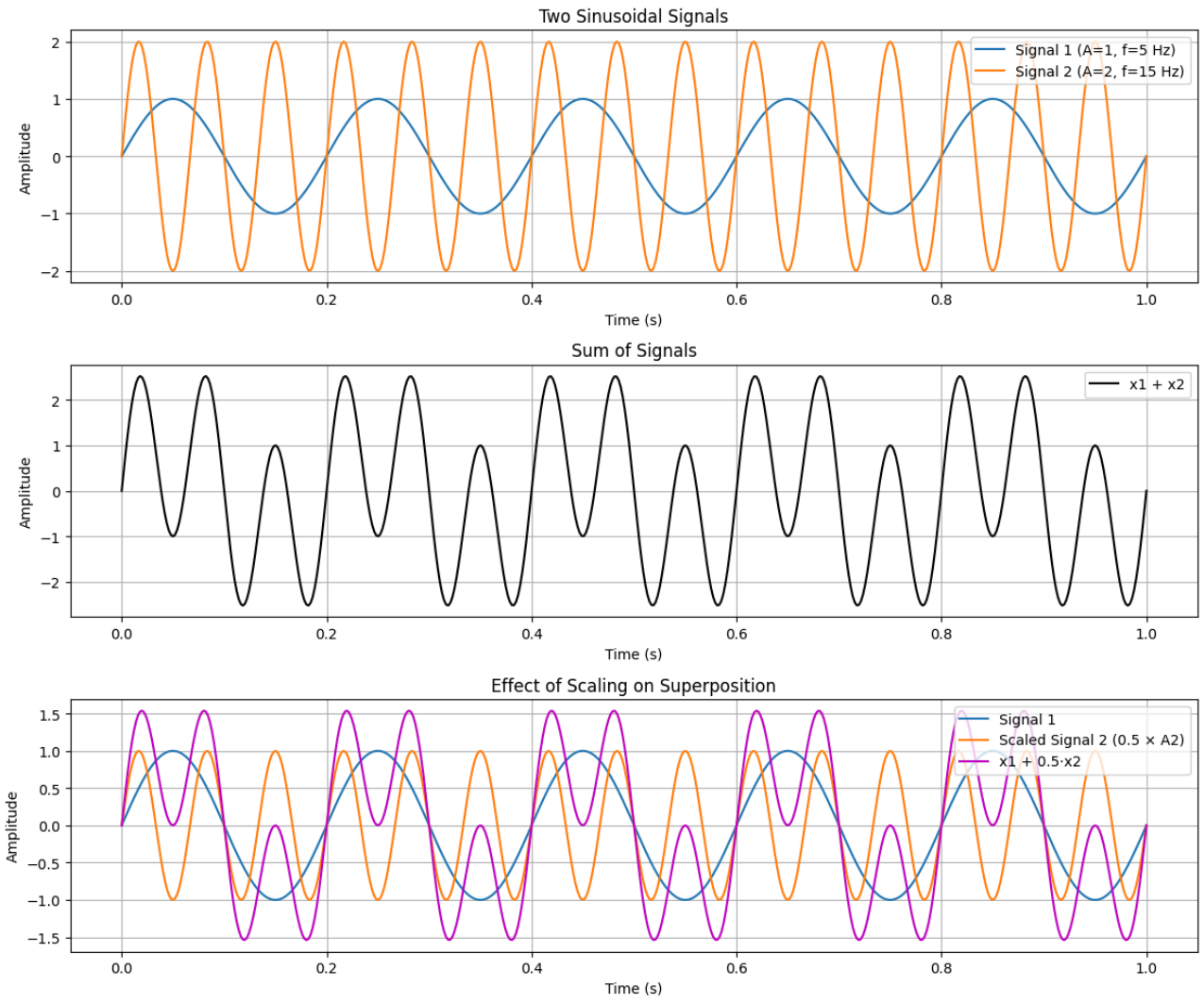
```
x1 = A1 * np.sin(2 * np.pi * f1 * t)
x2 = A2 * np.sin(2 * np.pi * f2 * t)
```

```
# (b) Add the signals
x_sum = x1 + x2
```

```
# (c) Scale one of the signals (e.g., 0.5 * x2)
x2_scaled = 0.5 * x2
x_sum_scaled = x1 + x2_scaled
```

```
# Plotting
plt.figure(figsize=(12, 10))

# Original signals
plt.subplot(3,1,1)
plt.plot(t, x1, label="Signal 1 (A=1, f=5 Hz)")
plt.plot(t, x2, label="Signal 2 (A=2, f=15 Hz)")
plt.title("Two Sinusoidal Signals")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)

# Summed signals
plt.subplot(3,1,2)
```

```
plt.plot(t, x_sum, 'k', label="x1 + x2")
plt.title("Sum of Signals")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)

# After scaling second signal
plt.subplot(3,1,3)
plt.plot(t, x1, label="Signal 1")
plt.plot(t, x2_scaled, label="Scaled Signal 2 (0.5 × A2)")
plt.plot(t, x_sum_scaled, 'm', label="x1 + 0.5·x2")
plt.title("Effect of Scaling on Superposition")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```




```
# (a) Generate clean sinusoidal signal
fs = 500              # Sampling frequency (Hz)
t = np.linspace(0, 1, fs, endpoint=False)  # 1 second duration
f = 5                 # Frequency of sinusoid
x_clean = np.sin(2 * np.pi * f * t)
```

```
# (b) Add Gaussian noise
noise = np.random.normal(0, 0.5, len(t))  # mean=0, std=0.5
x_noisy = x_clean + noise
```

```
# (c) Design Low-pass Butterworth filter
from scipy.signal import butter, lfilter

def butter_lowpass(cutoff, fs, order=4):
    nyq = 0.5 * fs          # Nyquist frequency
    normal_cutoff = cutoff / nyq
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
    return b, a
```

```
def lowpass_filter(data, cutoff, fs, order=4):
    b, a = butter_lowpass(cutoff, fs, order=order)
    y = lfilter(b, a, data)
    return y
```

```
# Apply filter (cutoff = 10 Hz, since signal is 5 Hz)
cutoff = 10
x_filtered = lowpass_filter(x_noisy, cutoff, fs)
```

```
# Plotting
plt.figure(figsize=(12,8))

# Clean signal
plt.subplot(3,1,1)
plt.plot(t, x_clean, 'b')
plt.title("Clean Sinusoidal Signal (5 Hz)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)

# Noisy signal
plt.subplot(3,1,2)
plt.plot(t, x_noisy, 'r')
plt.title("Noisy Signal (with Gaussian Noise)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)

# Filtered signal
plt.subplot(3,1,3)
plt.plot(t, x_noisy, 'r', alpha=0.5, label="Noisy")
plt.plot(t, x_filtered, 'g', linewidth=2, label="Filtered")
plt.plot(t, x_clean, 'b--', label="Original Clean")
plt.title("Filtered Signal (Low-pass Butterworth)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```